

메인 루프에서 try-except 안에 if-elif-else 구문이 다 들어가야 하는 것 아닌가 헷갈려. 어떤 구조가 맞는 걸까?

0 0

프로그래밍을 처음 배우실 때 누구나 한 번쯤 고민하게 되는 아주 예리하고 훌륭한 질문입니다!

결론부터 말씀드리면, 지금 작성하신 것처럼 try-except 블록 밖에 if-elif-else 구문을 두는 것이 실무에서도 권장하는 훨씬 안전하고 좋은 구조(Best Practice)입니다.

그 이유를 크게 두 가지로 나누어 설명해 드릴게요.

1. try 블록은 에러가 의심되는 '최소한의 구역'만 감싸야 합니다

만약 try 안에 if-elif-else 전체와 그 안에 있는 여러 함수들 (insert_and_calculate_information 등)을 전부 넣어버리면 어떻게 될까요?

나중에 입력 오류가 아니라 함수 내부의 전혀 다른 곳에서 예상치 못한 에러(예: 계산 실수, 오타 등)가 터졌을 때, 파이썬이 어디서 고장이 났는지 정확히 알려주지 않고 무조건 except 구문으로 빠져버리게 됩니다. 이렇게 되면 "진짜 원인"을 찾아 고치는 디버깅 작업이 매우 힘들어집니다.

따라서 try 는 '사용자가 숫자가 아닌 문자를 입력할 위험이 있는 딱 그 한 줄'만 검사하는 수문장 역할로 남겨두는 것이 좋습니다.

2. continue 명령어가 아래로 가는 길을 완벽히 차단합니다

if-elif-else 구문이 try 밖에 무방비하게 있는 것 같지만, 사실 완벽하게 보호받고 있습니다. 바로 except ValueError: 안쪽 끝에 있는 continue 덕분입니다.

1. try 블록은 에러가 의심되는 '최소한의 구역'만 감싸야 합니다

만약 try 안에 if-elif-else 전체와 그 안에 있는 여러 함수들

(insert_and_calculate_information 등)을 전부 넣어버리면 어떻게 될까요?

나중에 입력 오류가 아니라 함수 내부의 전혀 다른 곳에서 예상치 못한 에러(예: 계산 실수, 오타 등)가 터졌을 때, 파이썬이 어디서 고장이 났는지 정확히 알려주지 않고 무조건 except 구문으로 빠져버리게 됩니다. 이렇게 되면 "진짜 원인"을 찾아 고치는 디버깅 작업이 매우 힘들어집니다.

따라서 try 는 '사용자가 숫자가 아닌 문자를 입력할 위험이 있는 딱 그 한 줄'만 검사하는 수문장 역할로 남겨두는 것이 좋습니다.

2. continue 명령어가 아래로 가는 길을 완벽히 차단합니다

if-elif-else 구문이 try 밖에 무방비하게 있는 것 같지만, 사실 완벽하게 보호받고 있습니다. 바로 except ValueError: 안쪽 끝에 있는 continue 덕분입니다.

- **정상적으로 숫자를 입력하면:** try 를 무사히 통과했으니 except 는 무시하고 자연스럽게 아래의 if-elif-else 길로 진입합니다.
- **문자를 잘못 입력하면:** 즉시 에러가 터지면서 except 로 이동합니다. 경고 메시지를 띄운 후 continue 를 만나게 되는데, 이 continue 는 "더 이상 밑으로 내려가지 말고, 당장 반복문의 맨 처음 (메뉴 출력)으로 강제로 돌아가!"라는 강력한 지시를 내립니다. 잘못된 값이 if 문 쪽으로 흘러가는 것을 원천 봉쇄하는 것이죠.

이처럼 "입력을 검사하는 역할(try-except)"과 "선택된 메뉴를 실행하는 역할(if-elif-else)"을 깔끔하게 분리해 두었기 때문에, 현재 코드는 논리적으로 아주 훌륭한 구조를 갖추고 있습니다.